



Chapter 7 - OpenShift Logging



X sections in this chapter:

Section 7.1: Log Forwarding

Section 7.2: Guided Exercise

Section 7.3: Centralized Loggin

Section 7.4: Guided Exercise

Section 7.5: Lab

Section 7.6: Summary

Log Forwarding



Chapter 7 Section 1



Objectives

After completing this section, you will be:

- Deploy OpenShift Logging
 - for forwarding logs to an external aggregator.

Type of logs

Infrastructure logs: Container logs from pods that run in the openshift*, kube*, or default projects and journal logs sourced from node file system.

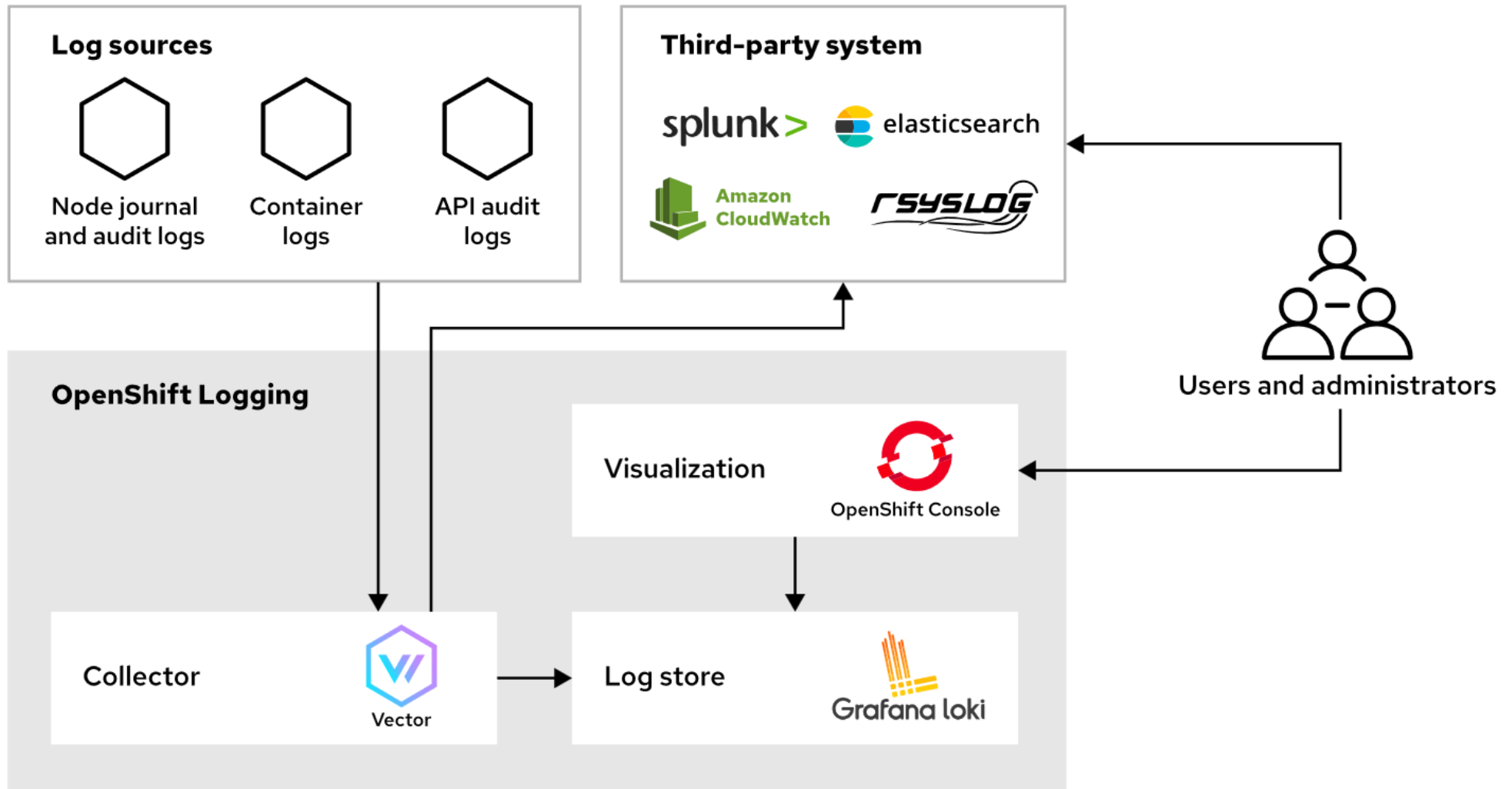
Audit logs: generated by the node audit system, auditd, Kubernetes API server, OpenShift API server, and OVN network.

Application logs: Container logs generated by user applications running in the cluster, except infrastructure container applications.

OpenShift Logging

- Collects and aggregates log messages from all pods and nodes.
- Use console to search and view log entries.
- Over years, logs can consume:
 - disk space.
 - compute resources.
- Bad solution:
 - Keep increasing disk space.
 - Keep adding more compute nodes.
- Better solution:
 - Forward logs to third-party log aggregator.
 - Forward logs to observability platform
- Example of third-party logging solutions:
 - Elasticsearch
 - AWS CloudWatch
 - Grafana Loki
 - Google Cloud Logging
 - Splunk

OpenShift Logging Components



Grafana Loki

- Aggregate and store logs from entire cluster in centralized location.
- Replaces Elasticsearch. (used in earlier OpenShift version)
 - Can install and use Elasticsearch via ECK Operator.
- Logs can be visualized thru
 - Built-in OpenShift Logging UI.
 - External vendor : Splunk, Grafana Cloud, Kibana.

Configure OpenShift Logging Components (1)

- The basic ClusterLogging YAML definitions:

```
apiVersion: logging.openshift.io/v1
kind: ClusterLogging
metadata:
  name: instance
  namespace: openshift-logging
spec:
  managementState: Managed
  collection:
    type: vector 3
```

- More advanced ClusterLogging YAML definitions (nxt)

Configure OpenShift Logging Components (2)

- More advanced ClusterLogging YAML definitions

```
apiVersion: logging.openshift.io/v1
kind: ClusterLogging
metadata:
  name: instance
  namespace: openshift-logging
spec:
  managementState: Managed
  collection:
    type: vector
  tolerations:
    - effect: NoSchedule
      key: node-role.kubernetes.io/infra
      value: reserved
    - effect: NoExecute
      key: node-role.kubernetes.io/infra
      value: reserved
```

Configure Log Collection and Forwarding (1)

- Default, forward all logs to internal log store (Loki).
- Can forward to third-party logging system:
- Example – Using external Splunk instance.

```
apiVersion: logging.openshift.io/v1
kind: ClusterLogForwarder
...output omitted...
spec:
  outputs:
    - name: splunk-receiver
      secret:
        name: splunk-auth-token
      type: splunk
      url: https://mysplunkserver.example.com:8088/services/collector
  pipelines:
    - name: to-splunk
      inputRefs: { audit, infrastructure }
...output omitted...
```

Configure Log Collection and Forwarding (2)

spec:

outputs:

- name: audit-rsyslog

syslog:

appName: ocp-prod

facility: local0

msgID: audit-msg

procID: audit-proc

severity: informational

rfc: RFC5424

type: syslog

url: tcp://audit-store.example.com:5514

- name: splunk-receiver

secret:

name: splunk-auth-token

type: splunk

url: https://mysplunkserver...

pipelines:

- name: audit-to-syslog

inputRefs:

- audit

outputRefs:

- audit-rsyslog

- name: admin-logs

inputRefs:

- audit

- infrastructure

outputRefs:

- default

- name: app-logs

inputRefs:

- application

outputRefs:

- splunk-receiver

Filtering Log (1)

- Limit number of logs to be forwarded.

```
spec:
  inputs:
  - name: production-apps
    application:
      selector:
        matchLabels:
          environment: production
  - name: qa-chain
    application:
      selector:
        matchLabels:
          environment: development
        namespaces:
          - qa-testing
          - builders
    ...output omitted...
```

```
pipelines:
  - name: to-splunk
    inputRefs:
      - qa-chain
      - production-apps
    outputRefs:
      - splunk-receiver
```

Filtering Log (2)

```
spec:
  filters:
    - name: unwanted-events
      type: kubeAPIAudit
      kubeAPIAudit:
        rules:
          - level: None
            namespaces:
              - openshift-*
              - kube*
            userGroups:
              - system:serviceaccounts:openshift-*
              - system:nodes
          - level: None
            users:
              - system:kube*
              - system:apiserver
            ...output omitted...

  pipelines:
    - name: audit-to-syslog
      inputRefs:
        - audit
      filterRefs:
        - unwanted-events
      outputRefs:
        - audit-rsyslog
```


Troubleshoot Log Forwarding

- The OpenShift Logging operator validates each log input, output, and pipeline for configuration errors:

```
$ oc -n openshift-logging describe clusterlogforwarder/instance
```

Status:

Conditions:

Message: **clusterlogforwarder is not ready**

Reason: **ValidationFailure**

...output omitted...

Critical - Apps - Syslog:

Last Transition Time: 2024-01-18T14:24:20Z

Message: **invalid: unrecognized outputs: [app-syslog], no valid outputs**

Reason: **Invalid**

Status: **False**

Type: **Ready**

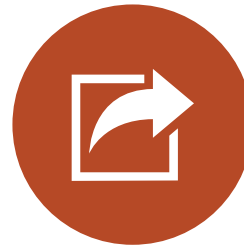
...output omitted...

Section summary

In this section, you have learned:



Reviewed components in OpenShift Logging components



Configure log collection and forwarding



Filtering logs



Troubleshoot misconfiguration

Section 7.2: Guided Exercise Log Forwarding

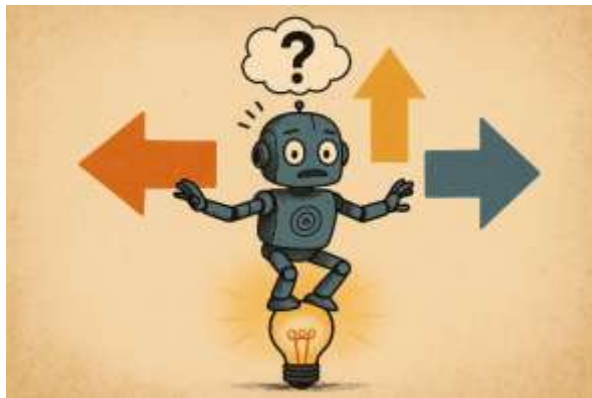
You should be able to:

- Deploy the OpenShift Logging operator.
- Configure the OpenShift Logging operator to forward logs to a remote syslog.

Estimated completion time: 25 - 35 minutes



**Continue /
Guided Exercise /
Break?**





Centralized Logging

Chapter 6 Section 2



Objectives

After completing this section, you will be:

- View alerting rules.
- Configure and silence alerts.

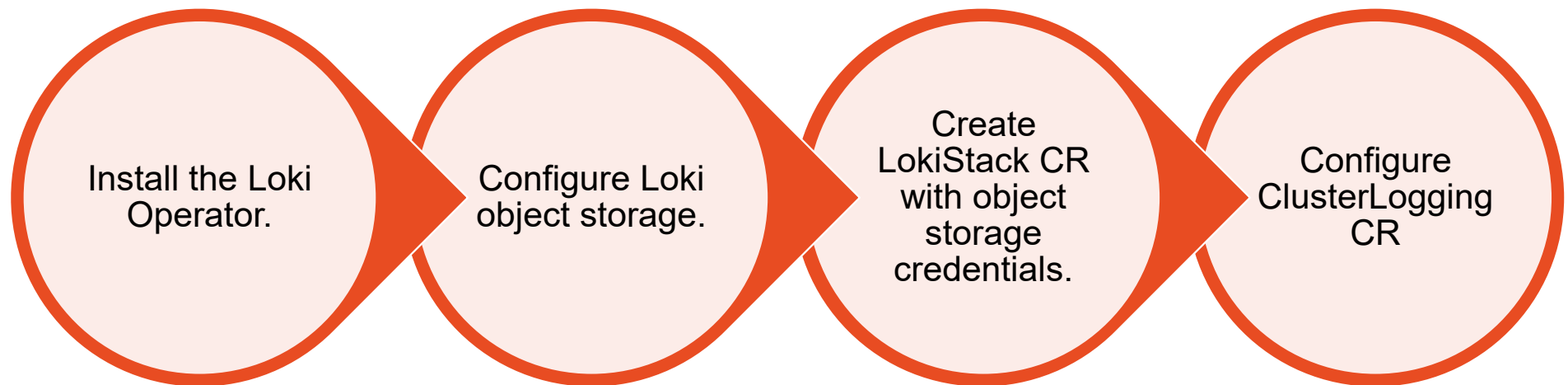
Log Retention

- By default, OpenShift :
 - stores logs from pods in local disk on compute nodes.
 - remove logs when pods are deleted.
 - remove logs when nodes get restarted.
 - execute garbage collectors: delete pods when disk space is needed.
- Logs are required to :
 - debug applications when there are issues.
 - analyze application lag or slow performance.
 - review object state changes.
- OpenShift Logging uses Grafana Loki as:
 - Internal log storage
 - to aggregate logs from entire cluster
 - centralized access control to logs.
 - integrates with external logs stores (Splunk, AWS CloudWatch, Google Cloud Logging)

Loki Log Store

- Is log aggregation tool from Grafana labs.
- Is horizontally scalable, highly available.
- Is Multitenant log aggregation system.
- Indexes logs, compresses logs and then stores log in chunk of object stores.
- Is managed by Loki operator
- Loki Operator supports object stores:
 - AWS S3, Azure Blob, Google Cloud Storage and OpenShift Data Foundation (ODF).
 - Includes LokiStack CR to manage Loki deployment.

Workflow for using Loki as log store



1. Install the Loki Operator

- From OperatorHub (web console)

Or

- From oc command line interface (CLI)

OperatorHub

Discover Operators from the Kubernetes community and Red Hat partners, curated by Red Hat. You can purchase commercial software through [Red Hat Marketplace](#). You can integrate services to your developers. After installation, the Operator capabilities will appear in the [Developer Catalog](#) providing a self-service experience.

The screenshot shows the OperatorHub web console interface. On the left is a sidebar with a list of categories: All Items, AI/Machine Learning, Application Runtime, Big Data, Cloud Provider, Database, Developer Tools, Development Tools, Drivers and plugins, Integration & Delivery, Logging & Tracing, Modernization & Migration, and Monitoring. The main area has a search bar with 'loki' entered. Below the search bar, three operator cards are displayed:

- Community Loki Operator** (provided by Grafana Loki SIG Operator): The Community Loki Operator provides Kubernetes native deployment and management o...
- Loki Helm Operator** (provided by opdev): An experimental operator that installs loki from its official helmchart.
- Loki Operator** (provided by Red Hat): The Loki Operator for OCP provides a means for configuring and managing a Loki stack for...

Configure Loki Object Storage

- Can use AWS S3, Azure Blob, Google Cloud Storage or Open Data Foundation.
- Course focuses on using ODF.

- ```
apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
 name: loki-bucket-odf
 namespace: openshift-logging
spec:
 generateBucketName: loki-bucket-odf
 storageClassName: openshift-storage.noobaa.io
```

# ODF create config map with bucket information

- Retrieve bucket information from configmap

```
$ oc get configmap/loki-bucket-odf -o yaml
```

```
apiVersion: v1
data:
 BUCKET_HOST: s3.openshift-storage.svc
 BUCKET_NAME: loki-bucket-odf-69d9...ab91
 BUCKET_PORT: "443"
 BUCKET_REGION: ""
 BUCKET_SUBREGION: ""
kind: ConfigMap
...output omitted...
```

# Create secrets with credential to access bucket

```
$ oc extract --to=- secret/loki-bucket-odf -n openshift-logging
```

```
AWS_ACCESS_KEY_ID
o0sOY8oaZtPfP18DRjSa
AWS_SECRET_ACCESS_KEY
z1QtDH37IUvLnkjpE3E4aS8yQl56CLPozGJOt31e
```

```
$ oc create secret generic logging-loki-odf \
 -n openshift-logging \
 --from-literal=access_key_id=o0sOY8oaZtPfP18DRjSa \
 --from-literal=access_key_secret=z1QtDH37IU...ozGJOt31e \
 --from-literal=bucketnames=loki-bucket-odf-69d9...ab91
```

# View, Search, and Query Logs

Project: openshift-logging ▼

[Installed Operators](#) > Operator details



**Red Hat OpenShift Logging**

5.8.0 provided by Red Hat, Inc

Actions ▼

[Details](#) [YAML](#) [Subscription](#) [Events](#) [All instances](#) [Cluster Log Forwarder](#) [Cluster Logging](#)

## Provided APIs

### **CLF** Cluster Log Forwarder

ClusterLogForwarder is an API to configure forwarding logs. You configure forwarding by specifying a list of [pipelines](#), which forward from a set of named inputs to a set of named outputs. There are built-in input names for common log categories, and...

### **CL** Cluster Logging

A Red Hat OpenShift Logging instance. ClusterLogging is the Schema for the clusterloggings API

### Provider

Red Hat, Inc

### Created at

Jan 23, 2024, 10:45 AM

### Console plugin

[Enabled](#)

### Links

Elastic

<https://www.elastic.co/>

# Filter logs in the web console

The screenshot shows the Kubernetes Logs web console interface. The top section is titled "Logs" and includes a "Show Histogram" link. A red box labeled "1" highlights the "Last 1 day" time range selector. To its right is a "Refresh off" toggle and a refresh button. Below this, a red box labeled "2" highlights the "Content" filter dropdown, which is set to "Search by Content". A red box labeled "3" highlights the "Severity" filter dropdown, which is set to "Severity". A red box labeled "4" highlights the "infrastructure" filter dropdown. A red box labeled "5" highlights the query input field containing the query `{ log_type="infrastructure" } | json`. To the right of the query field is a "Run Query" button. Below the filters is a "Hide Query" link. The bottom section displays a table of log results with two columns: "Date" and "Message".

| Date                         | Message                                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| > Jan 22, 2024, 10:09:11.750 | I0122 15:09:11.713637 1 log.go:194] reconciling (monitoring.coreos.com/v1, Kind=ServiceMonitor) openshift-multus/monitor-network       |
| > Jan 22, 2024, 10:09:11.750 | I0122 15:09:11.713489 1 log.go:194] Apply / Create of (apps/v1, Kind=DaemonSet) openshift-multus/network-metrics-daemon was successful |
| > Jan 22, 2024, 10:09:11.618 | I0122 15:09:11.510221 1 log.go:194] reconciling (apps/v1, Kind=DaemonSet) openshift-multus/network-metrics-daemon                      |

# LogQL Queries

- Logs are organized into streams called `chunks`.
- `Chunk` is sequence of log entries with similar set of labels.
- `Label` can include information such as name, environment, severity or any other relevant metadata for filtering and searching.
- LogQL is used as the query language for advanced filtering and searching logs with labels.
- Example 1 : Search for infrastructure logs only.

```
log_type="infrastructure" | json
```

- Example 2 : Filters the audit logs only from audit.log files in any directory:

```
log_type="audit" | json | file=~".*audit.log"
```

- Example 3 : Filters the audit logs that contain the error string but not the warning string, and that return a 403 or 503 status.

```
log_type="audit" | json |= "error" != "warning" |~ "status [45]03"
```



# Log Access Permissions

- Default, only administrator users can access.
- Assign following roles to user for access:
  - cluster-logging-application-view:** grants read permission to application logs.
  - cluster-logging-infrastructure-view:** grants read permission to infrastructure logs.
  - cluster-logging-audit-view:** grants read permission to audit logs

# Section summary

In this section, you have learned:



About Log Retention



Configure LokiStack



Using web console to view, search and query log



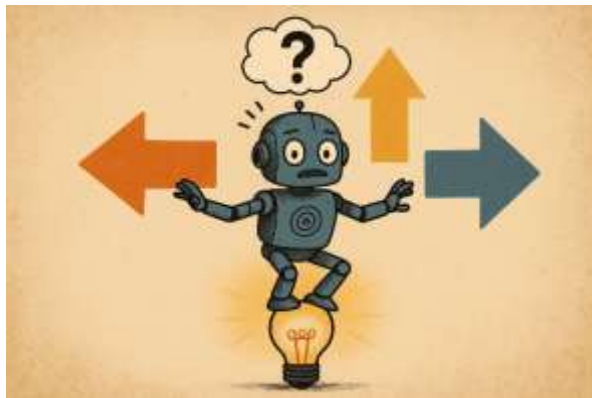
Using LogQL for advanced filtering



Grant access to log



**Continue /  
Guided Exercise /  
Break?**

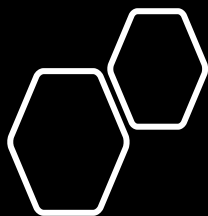


## **Section 7.4: Guided Exercise - Centralized Logging**

**You should be able to:**

- Use Loki as the log store for OpenShift Logging.
- Use Vector as the collector and the OpenShift web UI for log visualization.

**Estimated completion time:** 10 - 15 minutes



## Section 6.5: Lab OpenShift Logging

You should be able to:

Configure OpenShift Logging to forward logs to an external aggregator for long-term storage.

Configure OpenShift Logging with Loki for short-term log retention.

Configure OpenShift Logging to collect logs from specific applications.